AMCJワークショップ
共鳴するコード、SuperColliderで創る音の世界
- 実践とその先へ

2026年3月14日

田所淳

03: 「その先」3
WebブラウザーでSuperColliderを動かす!
Super Sonic

Super Sonicとは？

Super Sonicとは？

SUPERSONIC

SuperSonicは、SuperColliderの強力なオーディオ合成エンジンであるscsynthを、AudioWorkletとしてブラウザ上で動作させるプロジェクト
→ **つまりWebブラウザ上でSuperColliderのコードを実行できる!**

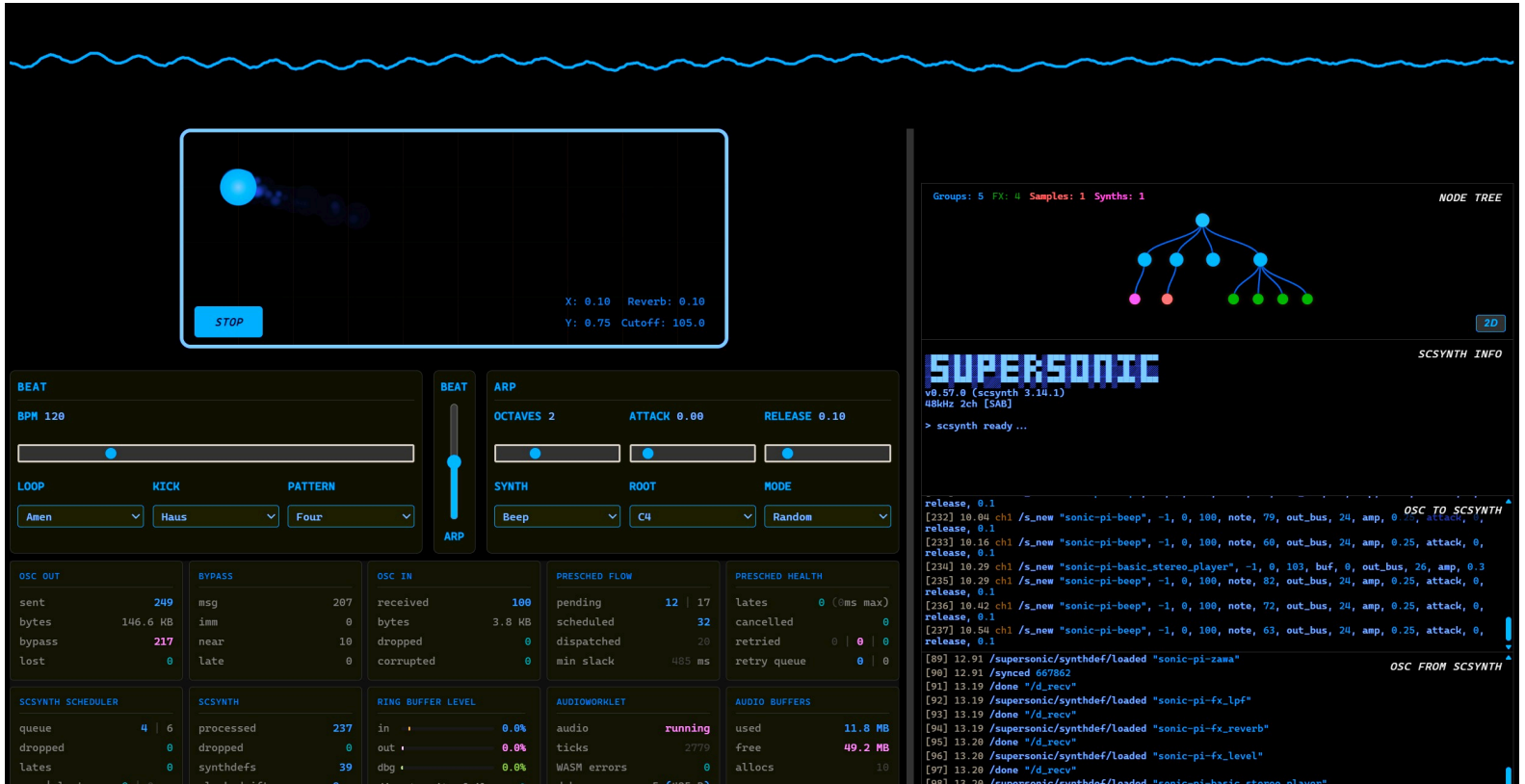
Sonic Piの開発者としても知られるSam Aaron氏が中心となって開発

Super Sonicとは？

まずはデモを試してみよう!

https://sonic-pi.net/supersonic/demo.html

すごい!!



Super Sonicセットアップ

Super Sonicセットアップ

今回は、ローカル環境にVSCodeからhttpサーバーを起動してSuper Sonicを動かす方法を紹介します

まずは、VScodeの拡張機能「Live Server」をインストール

- VSCodeの拡張機能から「Live Server」を検索してインストール
- インストール後、VSCodeの右下に「Go Live」というボタンが表示される

Super Sonicセットアップ

いよいよ、Super Sonicをセットアップしていきましょう!
Super Sonicを動かす方法は3つ。

1. CDN: クイックな実験、プロトタイプ、導入
2. npm: バンドラーを使用したJavaScriptプロジェクト
3. セルフホスト: 完全な制御、オフライン利用、本番環境へのデプロイ

Live Serverだと、CDNとnpmがなぜかうまくいかない…
今回はセルフホストでセットアップしてみましょう!

Super Sonicセットアップ

1. Super SonicのGihthubのリリースページから最新のリリースをダウンロード
   https://github.com/samaaron/supersonic/releases
2. ダウンロードしたzipファイルを解凍した「supersonic」フォルダを今回のワークショップのサンプルファイルの「supersonic-examples」フォルダ以下に移動

- AMCJ26
  - supersonic-examples
    - index.html
    - synth.html
    - simple.html
    - supersonic (ここに解凍したsupersonicフォルダを移動)

Super Sonicセットアップ

simple.htmlをブラウザで開いてみましょう!

bootボタンを押した後、triggerボタンを押す
→ シンセサイザー (Prophet 5) の音が再生される!

これでSuper Sonicのセットアップは完了です!

Super Sonicクィックスタート

まずは、simple.htmlを理解してみましょう!

`<script type="module">` から `</script>` までが重要な部分です

Super Sonicクィックスタート

まず前半部分

```
<script type="module">
      // SuperSonic モジュールをローカルファイルからインポート
      import { SuperSonic } from "./supersonic/supersonic.js";

      // SuperSonic インスタンスを生成
      // baseURL: SuperSonic 本体ファイルの場所
      // synthdefBaseURL: SynthDef（音色定義ファイル）の場所
      const supersonic = new SuperSonic({
        baseURL: "./supersonic/",
        synthdefBaseURL: "./supersonic/synthdefs/",
      });
```

SuperSonicクラスをsupersonic.jsからインポートして、Supersonicのインスタンスを生成しています

Super Sonicクィックスタート

つづき

```
// ボタン要素を取得
const bootBtn = document.getElementById("boot-btn");
const trigBtn = document.getElementById("trig-btn");

// boot ボタン: SuperCollider サーバーを起動し SynthDef を読み込む
bootBtn.onclick = async () => {
  await supersonic.init();                          // SuperCollider サーバーを非同期で起動
  await supersonic.loadSynthDef("sonic-pi-prophet"); // prophet シンセの SynthDef を読み込む
  // Metrics コンポーネントを SuperSonic に接続し、10fps でサーバー状態を更新
  document.getElementById("metrics").connect(supersonic, { refreshRate: 10 });
};
```

ここでは、bootボタンをクリックしたときの処理を定義しています

Super Sonicクィックスタート

つづき

```
  // trigger ボタン: OSC メッセージ /s_new でシンセを鳴らす
  // 引数: SynthDef名, ノードID(-1=自動), addAction, targetID, パラメータ...
  // note:52(ミ4), release:8(秒), cutoff:70(フィルターカットオフ)
  trigBtn.onclick = () => {
    supersonic.send("/s_new", "sonic-pi-prophet",
      -1, 0, 0, "note", 52, "release", 8, "cutoff", 70);
  };
</script>
```

triggerボタンをクリックで、OSCメッセージ/s_newを送信して、
sonic-pi-prophetというSynthDefを鳴らすようにしています

別のSynthDefを鳴らしてみよう!

先程のソースの「sonic-pi-prophet」を別のSynthDefの名称に変更すると別のシンセを演奏できる

では、どのようなSynthDefがあるのか?
supersonic/synthdefs/以下に定義されている

```
sonic-pi-beep.scsyndef
sonic-pi-blade.scsyndef
sonic-pi-bnoise.scsyndef
...
```

では、この中から、sonic-pi-tb303.scsyndefを鳴らしてみましょう!

Super Sonicクィックスタート

変更する場所は2箇所

```
...(中略)...
bootBtn.onclick = async () => {
  await supersonic.init();
  // ここを変更 sonic-pi-prophet → sonic-pi-tb303
  await supersonic.loadSynthDef("sonic-pi-tb303");
  document.getElementById("metrics").connect(supersonic, { refreshRate: 10 });
};

trigBtn.onclick = () => {
  // ここを変更 sonic-pi-prophet → sonic-pi-tb303
  supersonic.send("/s_new", "sonic-pi-tb303", -1, 0, 0, "note", 40, "release", 8, "cutoff", 90);
};
...(中略)...
```

音色が変化したはず!